

ASSESSMENT TOOL 1 (HIGH) – Analytical Problem Solving

NAME : S .BHUVANESH

REG NO : 192521004

1. Consider a linked list representing the sales of different products in a store for a week. Each node of the linked list contains the product ID and the number of units sold on a specific day. Implement a linked list data structure and provide functions to: Insert new sales data for a given product on a specific day. Calculate the total sales for each product at the end of the week. Find the product with the highest sales and its corresponding day. Display the sales data for a specific product for the entire week. Use your own data for product..

C Program

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int product, day, sales;
6      struct Node *next;
7  };
8
9  struct Node *head = NULL;
10
11 void insert(int p, int d, int s) {
12     struct Node *newNode = malloc(sizeof(struct Node));
13
14     newNode->product = p;
15     newNode->day = d;
16     newNode->sales = s;
17     newNode->next = head;
18     head = newNode;
19 }
20
21 void totalSales() {
22     struct Node *t;
23     int p, total;
24
25     for (p = 1; p <= 3; p++) {
26         total = 0;
27         t = head;
28
29         while (t != NULL) {
30             if (t->product == p)
31                 total += t->sales;
32             t = t->next;
33         }
34
35         printf("Product %d Total = %d\n", p, total);
36     }
37 }
```

```

38
39 void highestSale() {
40     struct Node *t = head, *max = head;
41
42     while (t != NULL) {
43         if (t->sales > max->sales)
44             max = t;
45         t = t->next;
46     }
47
48     printf("Highest Sale: Product %d, Day %d, %d units\n",
49           max->product, max->day, max->sales);
50 }
51
52 void display(int p) {
53     struct Node *t = head;
54
55     printf("Sales of Product %d:\n", p);
56
57     while (t != NULL) {
58         if (t->product == p)
59             printf("Day %d = %d units\n", t->day, t->sales);
60         t = t->next;
61     }
62 }
63
64 int main() {
65
66     insert(1, 1, 20);
67     insert(1, 2, 25);
68     insert(1, 3, 30);
69     insert(1, 4, 15);
70     insert(1, 5, 35);
71     insert(1, 6, 20);
72     insert(1, 7, 25);
73

```

```

74     insert(2, 1, 15);
75     insert(2, 2, 30);
76     insert(2, 3, 20);
77     insert(2, 4, 40);
78     insert(2, 5, 25);
79     insert(2, 6, 35);
80     insert(2, 7, 30);
81
82     insert(3, 1, 10);
83     insert(3, 2, 20);
84     insert(3, 3, 15);
85     insert(3, 4, 25);
86     insert(3, 5, 30);
87     insert(3, 6, 20);
88     insert(3, 7, 15);
89
90     printf("TOTAL SALES\n");
91     totalSales();
92
93     printf("\nHIGHEST SALE\n");
94     highestSale();
95
96     printf("\nWEEKLY SALES\n");
97     display(2);
98
99     return 0;
100 }
101

```

OUTPUT:

```
TOTAL SALES
Product 1 Total = 170
Product 2 Total = 195
Product 3 Total = 135

HIGHEST SALE
Highest Sale: Product 2, Day 4, 40 units

WEEKLY SALES
Sales of Product 2:
Day 7 = 30 units
Day 6 = 35 units
Day 5 = 25 units
Day 4 = 40 units
Day 3 = 20 units
Day 2 = 30 units
Day 1 = 15 units
```

2.A calculator application receives the following expression: $(8+2)*(6-4)/2$

C program :

```
1  #include <stdio.h>
2
3  int main() {
4      int stack[10], top = -1;
5      int a, b;
6
7      stack[++top] = 8;
8      stack[++top] = 2;
9      b = stack[top--];
10     a = stack[top--];
11     stack[++top] = a + b;
12
13     stack[++top] = 6;
14     stack[++top] = 4;
15     b = stack[top--];
16     a = stack[top--];
17     stack[++top] = a - b;
18
19     b = stack[top--];
20     a = stack[top--];
21     stack[++top] = a * b;
22
23     stack[++top] = 2;
24     b = stack[top--];
25     a = stack[top--];
26     stack[++top] = a / b;
27
28     printf("Expression: (8+2)*(6-4)/2\n");
29     printf("Result = %d\n", stack[top]);
30
31     return 0;
32 }
33
```

OUTPUT:

```
Expression: (8+2)*(6-4)/2
Result = 10
```